



الجامعة الافتراضية السورية
SYRIAN VIRTUAL UNIVERSITY

مقرر قواعد بيانات NoSQL

الفصل الأول: مقدمة في قواعد
بيانات NoSQL

Introduction to NoSQL Databases

Information

Technology

Engineering



Powered by:



رقم الصفحة

العنوان

- | | |
|----|--------------------------------------|
| 5 | 1. خصائص قواعد بيانات NoSQL |
| 7 | 2. ضمان اتساق البيانات |
| 10 | 3. قواعد بيانات المفتاح - قيمة |
| 13 | 4. قواعد بيانات عائلات الأعمدة |
| 16 | 5. قواعد البيانات الموجهة بالمستندات |
| 20 | 6. قواعد بيانات الرسم البياني |
| 23 | 7. تمارين |

كلمات مفتاحية:

NoSQL، SQL، CAP، ACID، BASE، اتساق البيانات، تجزئة البيانات.

ملخص:

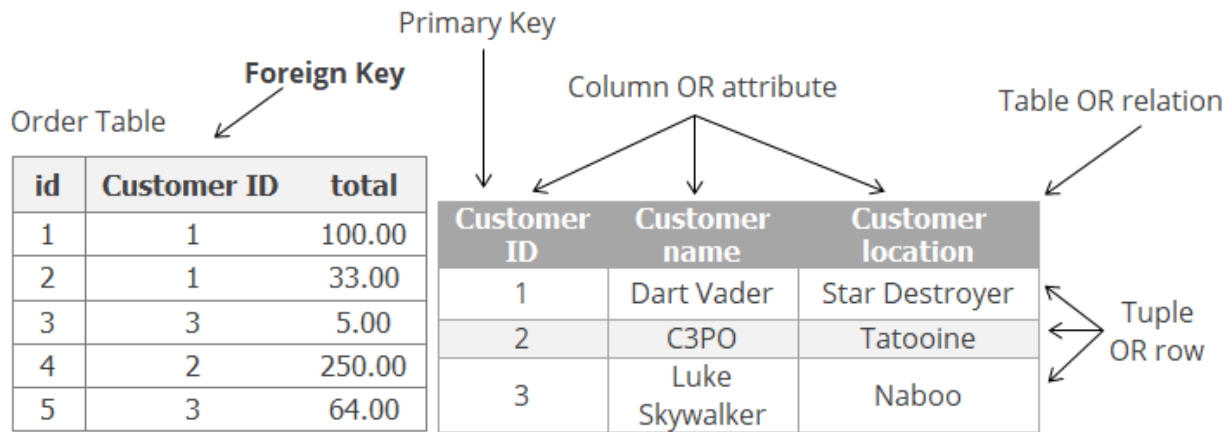
يُسلّط هذا الفصل الضوء على قواعد بيانات NoSQL وخصائصها وعيوبها وتطبيقاتها وأنواعها ومواصفات كل نوع منها، ويستعرض الفرق بينها وبين قواعد البيانات العلائقية. ويشتمل الفصل على مقارنة بين النموذجين ACID وBASE المتبعين لضمان اتساق قواعد البيانات، ويضم شرحاً لنظرية CAP.

أهداف تعليمية:

بعد انتهاء الطالب من دراسة الفصل الحالي، سيكون قادراً على فهم المواضيع الآتي ذكرها:

- خصائص قواعد بيانات NoSQL وعيوبها وتطبيقاتها
- الفرق بين قواعد بيانات NoSQL وقواعد البيانات العلائقية
- الفرق بين نموذج اتساق ACID ونموذج اتساق BASE
- نظرية CAP
- أنواع قواعد بيانات NoSQL ومواصفات كل نوع
- تجزئة البيانات في قواعد بيانات NoSQL
- التنسيق النهائي المتبع في قواعد بيانات NoSQL

منذ عدة عقود، يتم استخدام قواعد البيانات العلائقية (Relational Databases) لتخزين ما يوصف بالبيانات المنظمة (Structured Databases). حيث يتم تقسيم البيانات إلى مجموعات فرعية (انظر الشكل 1)، يشار إليها بالجدول (Tables). وتخزن الجداول وحدات بيانات محددة جيداً من حيث النوع والحجم وقيود أخرى، وتُعرف كل وحدة بيانات بالعمود (Column)، وتعبّر عن ميزة (أو صفة) واحدة لمجموعة البيانات ولذلك تُسمى أيضاً بالوصفة (Attribute). ولا يوجد تطابق بين أي سطرين من أسطر الجدول الواحد. وترتبط الأعمدة مع بعضها البعض بعلاقات (Relationships) كعلاقة أب-ابن، ويتم إنشاء هذه العلاقات بالاعتماد على شروط التكامل المرجعي الذي يعتمد بشكل أساسي على استخدام مفهوم المفتاح الأساسي (Primary Key) والمفتاح الأجنبي (Foreign Key).



الشكل 1: قاعدة بيانات علائقية

وعلى الرغم من وجود العديد من نماذج البيانات الأخرى كالنموذج الشبكي والهرمي وغيرها، إلا أن النموذج العلائقي تمكّن من الاحتفاظ بمركز الريادة في سوق قواعد البيانات لعشرات السنين، ومن أهم عوامل نجاحه اعتماده على لغة الـ SQL وهي لغة استعلام بسيطة ومعيّارية لهذا النموذج من قواعد البيانات (يوضح الشكل 2 مثلاً عنها). ويُعتبر الاتساق (Consistency) بين البيانات من أهم الشروط الواجب تحقيقها في قواعد البيانات العلائقية.

SQL Query

Result

2

```
SELECT customerID
FROM order
WHERE total > 200 ;
```

الشكل 2: مثال عن استعلام بلغة SQL

ومع التطور الكبير للتطبيقات وظهور منصات وتطبيقات ويب تتعامل مع حجوم ضخمة من البيانات في كل ثانية، كان لابد لهذه التطبيقات وقواعد البيانات المرتبطة بها من امتلاك المقدرة على التوسع (Scalability). وتُشير مقدرة تطبيق ما على التوسع إلى قابليته على استيعاب النمو المتزايد في عدد الاستعلامات التي ينفذها وحجم البيانات التي يعالجها وعدد الزبائن التي يستجيب لطلباتها مع المحافظة على أداء جيد ومن دون توقف التطبيق عن العمل وتقديم الخدمات المطلوبة منه. فهذا النمو يفرض تحديات أمام المخدم المشغّل لقاعدة البيانات التي يتعامل معها التطبيق. فتصبح وحدة المعالجة المركزية و/أو الذاكرة الرئيسية محملة بشكل زائد، وهذا ما يؤدي إلى زيادة زمن معالجة وزمن تنفيذ الاستعلامات، كما يزيد زمن تخزين البيانات عند زيادة حجم البيانات اللازم تخزينها. إضافة لذلك، تزداد الحمولة على الشبكة الناقلة للبيانات ويزداد استهلاك عرض حزمة نقل البيانات. وكل ما سبق ذكره سينعكس سلباً على أداء التطبيق ورضى مستخدمي التطبيق. ويمكن الحل لهذه المشكلة في تحسين الرمز البرمجي وتحسين الاستعلامات والتخزين المؤقت وفهرسة الاستعلامات بشكل مناسب. ولكن قد لا تكفي هذه التحسينات ويكون لا بد من زيادة الموارد المستخدمة من قبل مخدم قاعدة البيانات. وهنا لا بُدّ من التفكير بتوسيع قاعدة البيانات شاقولياً أو أفقياً من خلال إضافة موارد جديدة من أجل توزيع الحمل وتحسين الأداء.

ففي حالة التوسع الشاقولي (Vertical Scaling)، تتم زيادة القدرة الحاسوبية وحجم الذاكرة للمخدم من دون أي تأثير على قاعدة البيانات، ولكن هذه الزيادة لها حد أعلى متعلق بالكيان الصلب ككل، وهي مرهونة بتكلفة عالية لا تتناسب خطياً مع التحسن المتحصل عليه. أما في حالة التوسع الأفقي (Horizontal Scaling) فتتم إضافة مخدمات تمثل عقداً جديدة لتوزيع الحمل عليها، ومن الطبيعي أن يتم توزيع البيانات أيضاً على العقد المضافة. ولكن التوسع الأفقي لقواعد البيانات العلائقية مع المحافظة على الاتساق يُعتبر مهمة صعبة وقد تكون مستحيلة في بعض الحالات. وتفشل قواعد البيانات العلائقية في معالجة الاستعلامات في المنصات العالمية الضخمة (مثل Facebook و Google و Amazon) التي يردها الملايين من الاستعلامات في غضون فترة زمنية قصيرة. ولذلك ظهرت قواعد بيانات NoSQL لتأمين سرعة في معالجة هذه الاستعلامات ومرونة في تخزين البيانات الضخمة واسترجاعها.

1. خصائص قواعد بيانات NoSQL:

تم استخدام الاسم المختصر NoSQL لأول مرة في العام 1998 كتسمية لقواعد البيانات "العلائقية" البسيطة والمفتوحة المصدر والتي لا تستخدم SQL. وظهرت التسمية مرة أخرى في العام 2009 للدلالة على قواعد البيانات غير العلائقية، لأنه غالباً ما يُشار إلى قواعد البيانات العلائقية باسم قواعد بيانات SQL. وترمز الكلمة NoSQL إلى "No SQL"، أي "لا يوجد استخدام لـ SQL". ولكن الدلالة الأكثر شيوعاً "Not only SQL" أي "ليس فقط SQL" للتأكيد على حقيقة أن قواعد البيانات هذه قد تدعم لغات استعلام تشبه الـ SQL (لغة CQL مثال على ذلك)، والتأكيد على قدرتها على التعامل مع البيانات المنظمة وغير المنظمة على حدٍ سواء. وتطورت قواعد بيانات NoSQL كاستجابة إلى التوسع الحاصل في تطبيقات الويب واستخدامها لبيانات ضخمة غير منظمة (Unstructured) تحتاج إلى معالجة بأسرع وقت. وعادة ما تستخدم قواعد بيانات NoSQL نظم قواعد بيانات موزعة على عدة عقد (أجهزة حاسوبية)، وتعالج كميات كبيرة من أنواع مختلفة من البيانات. ولذلك يتم تفضيلها على قواعد البيانات العلائقية عند معالجة البيانات الضخمة غير المنظمة نظراً لسرعتها ومرونتها. وهذا ما أدى إلى الاعتماد عليها من قبل المنصات Facebook و Twitter و LinkedIn و Google وغيرها. حيث تقوم هذه المنصات بمعالجة كميات هائلة من البيانات غير المنظمة وتنسيقها للعثور على أنماط جديدة واكتساب رؤى تجارية جديدة. وسُمي هذا النوع من البيانات بـ "البيانات الضخمة" وتم إطلاقه كمصطلح رسمياً في العام 2005، ومنذ ذلك الحين توسعت الأبحاث حول هذا النوع من البيانات. وأنظمة قواعد البيانات العلائقية (أو قواعد بيانات SQL) هي أكثر بكثير من مجرد أنظمة لتخزين البيانات. فهي تمتلك قدرة عالية على القيام بمختلف عمليات معالجة البيانات، وتتمتع بالخصائص التالية:

- تمتلك لغة استعلام قوية (SQL)
- لها مخطط عام (Global Schema) وبيانات وصفية (Metadata) لتوصيف بنيتها، ويجب إنشاء المخطط العام قبل إدخال البيانات في قاعدة البيانات
- تضمن الاتساق (Consistency)
- تدعم مفهوم التكامل المرجعي
- تمتلك قابلية الاسترداد والتسجيل
- تمتلك إمكانية التشغيل والتزامن متعدد المستخدمين
- تسمح بالتحكم بالمستخدمين والأدوار والأمن
- تدعم الفهرسة

وهذه المزايا العديدة مطلوبة بقوة في الكثير من التطبيقات منها برمجيات المصارف وبرامج التأمين. ومع ذلك، نظراً لأن التحكم في تكامل البيانات يتطلب الكثير من العمل وقوة المعالجة، فإن قواعد البيانات العلائقية تصبح محدودة عند معالجة كميات كبيرة من البيانات. ونظم إدارة قواعد البيانات العلائقية لا تمتلك الكفاءة وسرعة الأداء والمرونة التي تتطلبها معالجة البيانات الضخمة.

ومن الناحية العملية، غالباً ما تعرقل عمليات الحفاظ على الاتساق المعالجة الفعالة للكميات الضخمة من البيانات، وخاصة في حالات الاستخدام التي يكون فيها التركيز على الأداء بدلاً من الاتساق كحالات وسائل التواصل الاجتماعي. وهذا هو السبب الذي دفع مطورو تطبيقات الويب والمصادر المفتوحة إلى تطوير أنظمة قواعد البيانات الموزعة الضخمة التي يمكنها تلبية هذه المتطلبات الجديدة.

ويعتمد القرار باستخدام قواعد بيانات NoSQL على المشكلة التي يجب حلها، وتتنصف ببساطة التصميم وسهولة التوسع أفقياً لضم عناقيد من الآلات التي يسهل التحكم بها بكثير من الدقة. وتختلف بُنى المعطيات المستخدمة بواسطة NoSQL عن تلك المستخدمة في قواعد البيانات العلائقية مما يجعل بعض العمليات أسرع في NoSQL. ويُنظر أحياناً إلى بُنى المعطيات المستخدمة بواسطة NoSQL على أنها أكثر مرونة من جداول قواعد البيانات العلائقية. وتتخلى العديد من قواعد بيانات NoSQL عن التناسق (Atomicity) لصالح التوافر (Availability) والسرعة (Speed) والتسامح مع التقسيم (Partition Tolerance).

وتتمتع قواعد بيانات NoSQL بالخصائص الآتية:

- نموذج البيانات ليس علائقياً
- تركيز على التوزيع الواسع وخاصة التوسع الأفقي
- وجود مخطط مسبق للبيانات ليس إلزامياً
- النسخ المتماثل للبيانات (Data Replication) سهل التطبيق
- وصول سهل للبيانات من خلال API
- نموذج الاتساق ليس بالضرورة ACID (غالباً BASE)

وتوجد الكثير من التحديات التي تحول دون التخلي التام عن قواعد البيانات العلائقية لصالح قواعد بيانات NoSQL التي تستخدم لغات استعلام منخفضة المستوى وغير معيارية، كما تعاني قواعد بيانات NoSQL من صغر حجم الاستثمارات الضخمة لها مقارنة مع الاستثمارات الضخمة في قواعد البيانات العلائقية. وتفتقر معظم قواعد بيانات NoSQL إلى تطبيق حقيقي لمتطلبات ACID1 الخاصة بالمعاملات، على الرغم من أن بعض قواعد البيانات، مثل MarkLogic و Aerospike و c-treeACE و Symas LMDB و OrientDB جعلت تطبيق هذه المتطلبات مركزي في تصاميمهم.

¹Atomicity, Consistency, Isolation, Durability

2. ضمان اتساق البيانات:

تقدم معظم قواعد بيانات NoSQL مفهوماً خاصاً للاتساق (Consistency) حيث يتم نشر التغييرات التي تطرأ على قاعدة البيانات على جميع العقد، ولذلك قد لا تقوم الاستعلامات الخاصة بالبيانات بإرجاع البيانات المحدثة على الفور أو قد تؤدي إلى قراءة بيانات غير دقيقة وهي مشكلة تُعرف بالقراءات القديمة (Stale Reads). كما قد تعرض بعض أنظمة NoSQL عمليات الكتابة المفقودة (Lost Writes) وأشكالاً أخرى من فقدان البيانات. وتوفر بعض أنظمة NoSQL مفاهيم مثل التسجيل المسبق (Write-ahead Logging) لتفادي فقدان البيانات. أما بالنسبة إلى معالجة المعاملات الموزعة على قواعد بيانات متعددة، فيُعد تناسق البيانات تحدياً أكبر. وهذا صعب لكل من قواعد البيانات العلائقية و NoSQL على حدٍ سواء. وحتى في قواعد البيانات العلائقية المتاحة حالياً، يصعب توسيع قواعد البيانات باستخدام قيود التكامل المرجعي. ويوجد عدد قليل من الأنظمة التي تحافظ على معايير X/Open XA الخاصة بمعالجة المعاملات الموزعة ومواصفات ACID المتعلقة بمعالجة المعاملات.

ويعد ضمان سلامة البيانات مطلباً رئيسياً للعديد من تطبيقات قواعد البيانات. وتسمح إدارة المعاملات (Transaction Management) لنظام قاعدة البيانات بالعمل المتزامن الخالي من التعارض من قبل العديد من المستخدمين. ويتم تطبيق التغييرات على قاعدة البيانات وتصبح مرئية فقط إذا تم استيفاء جميع قيود التكامل على النحو المحدد من قبل المستخدمين. ويصف مصطلح المعاملة (Transaction) عمليات قاعدة البيانات المرتبطة بقواعد التكامل المرجعي، والتي تقوم بتحديث حالات قاعدة البيانات مع الحفاظ على الاتساق. وبشكل أكثر تحديداً، فإن المعاملة هي سلسلة من العمليات التي يجب أن تكون ذرية (Atomic) ومتسقة (Consistent) ومعزولة (Isolated) ومتينة (Durable). وتُعرف هذه الخصائص الأربع بخصائص ACID وهي المسؤولة في حال تحققها عن ضمان اتساق البيانات.

وتعني كلمة "ذرية" أنه يتم تنفيذ كامل عمليات المعاملة بالكامل أو لا يتم على الإطلاق، وفي حال إجهاض المعاملة يجب أن يتم ذلك من دون ترك أي أثر من آثارها في قاعدة البيانات. ويجب أن تكون الحالات الوسيطة للمعاملة التي تم إنشاؤها بواسطة العمليات الفردية داخلها غير مرئية من قبل المعاملات المتزامنة الأخرى. ويجب أن تضمن المعاملة نقل قاعدة البيانات من حالة متسقة إلى حالة متسقة أخرى، وأثناء تنفيذ عمليات المعاملة تمر قاعدة البيانات بفترة مؤقتة من عدم الاتساق ولذلك يجب عزلها وعدم استخدام القيم المؤقتة للبيانات قبل انتهاء عملية تثبيت المناقلة. وفي حالة وجود أخطاء في البرامج أو تعطل النظام أو حدوث أخطاء في وسائط التخزين الخارجية، يمكن دوماً إعادة قاعدة البيانات إلى حالة متسقة لأنه يتم الاحتفاظ بجميع تأثيرات المعاملة من أجل استرجاع حالة الاتساق السابقة في حالات الفشل.

والكثير من المطورين ملم بمواصفات ACID التي تفرضها قواعد البيانات العلائقية، وتُعد بالنسبة لهم الضمان الحقيقي للحصول على بيئة آمنة لمعالجة البيانات. فبمجرد اكتمال أي معاملة، تكون بياناتها متسقة ومستقرة على الأقراص الصلبة. ويعد تناسق الكتابة أمراً رائعاً لمطوري التطبيقات، ولكنه يتطلب أيضاً قفلاً للبيانات والذي يكون عادةً مشكلة لمعظم حالات الاستخدام. وعندما يتعلق الأمر بتقنيات NoSQL، فإن معظم قواعد بيانات الرسم البياني (بما في ذلك Neo4j) تستخدم نموذج اتساق ACID لضمان أمان البيانات وتخزينها باستمرار.

وبالنسبة للعديد من المجالات وحالات الاستخدام، تكون معاملات ACID أكثر تشدداً بشأن اتساق البيانات مما يتطلبه المجال أو الحالة بالفعل. ومعظم قواعد بيانات NoSQL تعتبر معاملات ACID أقل عصرية، فقد خففت متطلبات الحصول الفوري على اتساق البيانات وتحديثها ودقتها من أجل الحصول على مزايا أخرى، مثل التعامل مع حجوم كبيرة من البيانات بمرونة. ولذلك اعتمدت نموذج الاتساق BASE، والذي يفرض توفر الخصائص التالية:

- **التوافر الأساسي (Basic Availability):** ويقضي أن تعمل قاعدة البيانات في غالبية الأوقات. وبدلاً من فرض الاتساق الفوري، سيتم ضمان توافر البيانات عن طريق نشرها وتكرارها عبر عقد عنقود قاعدة البيانات.

- **الحالة الناعمة (Soft-state):** ليس بالضرورة أن تكون مخازن البيانات متسقة أثناء الكتابة، ولا أن تكون النسخ المتماثلة المختلفة متسقة طوال الوقت. ونظراً لعدم وجود تناسق فوري، قد تتغير قيم البيانات بمرور الوقت. ولا يؤكد BASE على ضرورة أن تفرض قاعدة البيانات اتساقها، بل يفوض تلك المسؤولية للمطورين.

- **الاتساق النهائي (Eventual consistency):** تصبح مخازن البيانات متسقة في وقت لاحق لعملية الكتابة، وتظل قراءة البيانات ممكنة أثناء الكتابة على الرغم من أنها قد لا تعكس الواقع. ومن الملاحظ أن خصائص BASE أقل صرامة من ACID، فهي تركز على التوافر، ولا تقدم اتساقاً مضموناً للبيانات المنسوخة أثناء الكتابة، ولكنها تضمن أن تكون البيانات متسقة فيما بعد.

وتفضل شركات التسويق وخدمة العملاء التي تتعامل مع تحليل المشاعر مرونة BASE عند إجراء أبحاث الشبكات الاجتماعية الخاصة بهم. حيث يصعب تنظيم موجزات الشبكة الاجتماعية بشكل جيد، وتحتوي على كميات هائلة من البيانات التي يمكن لقاعدة البيانات المصممة على طراز BASE تخزينها بسهولة. وتميل قواعد بيانات NoSQL إلى التوافق مع مبادئ BASE، حيث تعد MongoDB و Cassandra و Redis من بين حلول NoSQL الأكثر شيوعاً التي تفضل نموذج اتساق BASE.

ولقد أصبح من الواضح من الناحية العملية أنه بالنسبة لأنظمة تخزين البيانات الضخمة والموزعة، لا يمكن أن يكون الاتساق هو الهدف الأساسي دائماً؛ ففي بعض الأحيان يكون التوافر والتسامح مع التقسيم أولوية أكبر. وتطبيقات الويب موجهة نحو التوافر العالي والقدرة على مواصلة العمل في حالة فشل إحدى

عقد الشبكة. وتستخدم أنظمة تحمل التقسيم هذه عقداً (أجهزة حاسوبية) مكررة ومتطلبات تناسق أكثر ليونة تسمح للعقد المنسوخة بالاحتفاظ بإصدارات متباينة من البيانات مؤقتاً ولا يتم تحديثها إلا بعد مرور فترة زمنية.

وخلال ندوة في العام 2000، قدم Eric Brewer من جامعة كاليفورنيا في بيركلي نظرية CAP التي مفادها أنه من المستحيل أن يتمتع أي نظام موزع بجميع الخصائص الثلاث معاً: الاتساق (Consistency) والتوافر الدائم (Availability) والتسامح مع التقسيم (Partition Tolerance)، ولذلك يجب الموازنة بين هذه الخصائص عند إنشاء أي نظام موزع. ومن الناحية العملية، يمكن الجمع بين الاتساق والتوافر (CA) أو بين الاتساق والتسامح مع التقسيم (CP) أو بين التوافر والتسامح مع التقسيم (AP). وكأمثلة على حالات استخدام لهذه النظرية:

- تتطلب أنظمة البورصة الاتساق والتوافر (CA)، وهي محققة بالفعل لأن هذه الأنظمة تستخدم قواعد البيانات العلائقية التي تتصف بخصائص ACID.
 - شبكات أجهزة الصراف الآلي (ATM) على مستوى البلد تتطلب الاتساق والتسامح مع التقسيم (CP)، وبما أن أوقات الاستجابة الطويلة نسبياً مقبولة، لذلك يمكن استخدام قواعد البيانات العلائقية الموزعة والمكررة أو أنظمة NoSQL التي تدعم CP مع هذه الشبكات.
 - تُستخدم خدمة الإنترنت نظام DNS لتحويل عناوين URL إلى عناوين IP رقمية لإنشاء اتصال وفقاً لبروتوكول TCP/IP، وبالتالي يجب أن تكون متاحة دائماً ومتسامحة مع التقسيم (AP)، وهي مهمة تتطلب قواعد بيانات NoSQL، نظراً لأن قواعد البيانات العلائقية لا يمكنها المحافظة على التوافر العالي العالمي والتسامح مع التقسيم في نفس الوقت.
- وقاعدة البيانات هي بنية يتم فيها تخزين البيانات، ويُطلق عليها أحياناً مخزن البيانات (Data Store) على الرغم من أن هذه التسمية أشمل، وأبسط مثال على مخزن البيانات هو الملفات المسطحة (Flat-Files) المخزنة على القرص الصلب والتي نتعامل معها باستمرار، وقد تكون نصية (ذات لاحقة csv أو txt أو tsv) أو مرمزة ثنائياً (Binary). ويتم اعتبار قاعدة البيانات نوعاً خاصاً من الملفات المسطحة، حيث يتم تخزينها فيزيائياً ضمن ملفات على أقراص صلبة، ولكنها تختلف عن الملفات المسطحة في أن قواعد البيانات تحتاج إلى نظام خاص لإنشائها وتخزين البيانات فيها واسترجاعها، ويُسمى هذا النظام بنظام إدارة قواعد البيانات. ولا تحتاج جميع قواعد بيانات NoSQL إلى نظام لإدارتها وإنما قد تتم الإدارة عن طريق التطبيق الذي يتعامل معها.
- ويوجد عدة أنواع من قواعد بيانات NoSQL، ويمكن في التطبيق الواحد استخدام عدة أنواع من أجل الاستفادة من نقاط القوة لكل نوع. وأشهر هذه الأنواع: قواعد بيانات المفتاح - قيمة وقواعد بيانات عائلات الأعمدة وقواعد البيانات الموجهة بالمستندات وقواعد بيانات الرسم البياني.

3. قواعد بيانات المفتاح - قيمة:

قواعد بيانات المفتاح - قيمة هي نموذج لتخزين البيانات مصمم لتخزين واسترجاع البيانات وإدارة بنية معطيات ثنائية أشبه ما تكون بالبنية المعروفة بـ "قاموس البيانات" (Data Dictionary). وتحتوي هذه البيئة على مجموعة من الكائنات، أو السجلات، والتي بدورها تحتوي على العديد من الحقول المختلفة بداخلها، وكل منها يحتوي على بيانات. ويتم تخزين هذه السجلات واستردادها باستخدام مفتاح يعرف السجل بشكل فريد، ويُستخدم للعثور على البيانات داخل قاعدة البيانات.

وتعمل قواعد بيانات المفتاح - قيمة بطريقة مختلفة تماماً عن قواعد البيانات العلائقية التي تحدد مسبقاً مخطط البيانات (Data Schema)، والذي بدوره يصف بنية تخزين البيانات بعدد من الجداول الحاوية على حقول ذات أنماط معطيات محددة جيداً. وفي المقابل، تتعامل قواعد بيانات المفتاح-قيمة مع البيانات على أنها مجموعة واحدة قد تحتوي على حقول مختلفة لكل سجل. أي ليس بالضرورة أن يكون لجميع السجلات نفس البنية. ويوفر هذا الأمر قدراً كبيراً من المرونة ويناسب أكثر المفاهيم الحديثة مثل البرمجة غرضية التوجه.

ويُعتبر تخصيص قيمة لمتغير أو مفتاح من أبسط الطرائق لتخزين البيانات. وعلى مستوى الأجهزة، تعمل وحدات المعالجة المركزية (CPU) مع السجلات بناءً على هذا النموذج؛ وتستخدم لغات البرمجة هذا المفهوم في المصفوفات الترابطية (Associative Arrays). ووفقاً لذلك، فإن أبسط نموذج ممكن لقاعدة البيانات هو تخزين كيان بيانات كقيمة لكيان بيانات آخر يُسمى المفتاح. وفي هذا النوع من قواعد البيانات، يمكن تخزين قيمة محددة لأي مفتاح باستخدام أمر بسيط. مثل الأمر SET:

```
SET User: U17547:firstname John
SET User:U17547:lastname Doe
SET User:U17547:email john.doe@blue_planet.net
SET User:U17547:pwhash D75872C818DC63BC1D87EA12
SET User:U17548:firstname Jane
SET User:U17548:lastname Doherty
...
```

ففي المثال السابق، يتم تخزين بيانات مستخدمي موقع الويب، وتضم هذه البيانات: الاسم الأول والاسم الأخير والبريد الإلكتروني وكلمة المرور المشفرة. فمثلاً يتم تخزين القيمة John للمفتاح U17547:firstname

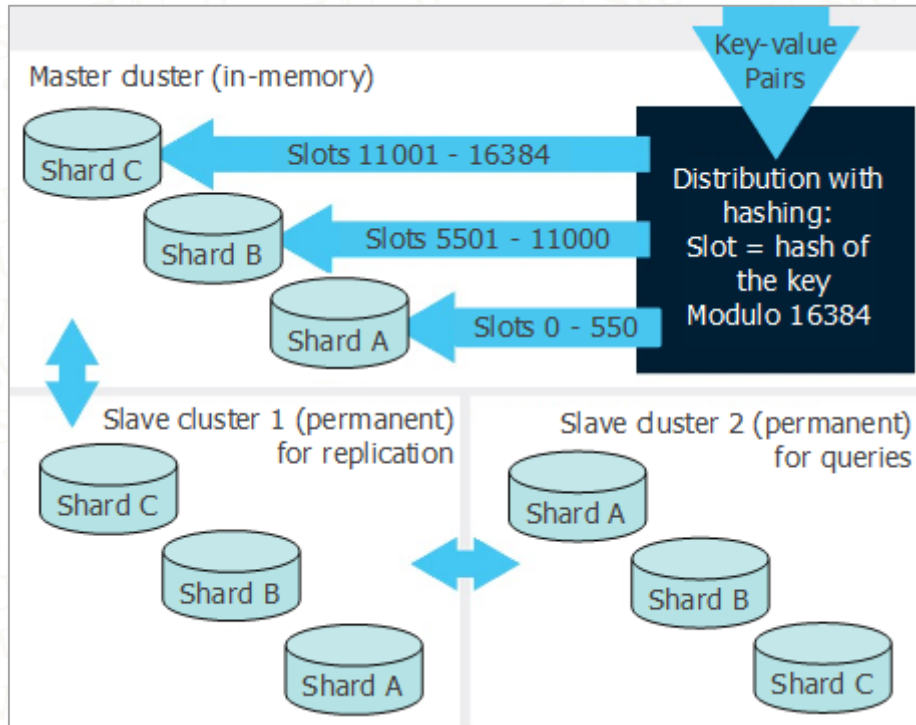
ويمكن استرجاع البيانات بالاعتماد على المفتاح كما يلي:

> GET User:U17547:email
john.doe@blue_planet.net

وعادة ما يتم بناء المفتاح باستخدام محارف خاصة مثل النقطتين (:). أو الشرطات المائلة (/). وهذا ما يسمح باستخدام محارف خاصة عند كتابة اسم المفتاح. ويمكن تخزين كائنات البيانات في أي وقت وباستخدام تنسيقات مختلفة من دون الحاجة إلى استخدام كائنات لتمثيل البيانات الوصفية كما في حالة قواعد البيانات العلائقية. وعدم وجود مخطط أو تكامل مرجعي يجعل قواعد بيانات المفتاح - قيمة فعالة في معالجة الاستعلامات (حيث يسهل تقسيمها) ومرنة فيما يتعلق بأنواع البيانات المراد تخزينها. وتتمتع قواعد بيانات المفتاح - قيمة بالخصائص التالية:

- تحتوي على مجموعة من المفاتيح، وهي عبارة عن كائنات البيانات التي تُستخدم لتحديد كائنات أخرى.
 - من أجل كل مفتاح، يوجد كائن بيانات واحد فقط مرتبط به، يُسمى قيمة هذا المفتاح.
 - تحديد مفتاح ما، يسمح بالاستعلام عن القيمة المرتبطة به في قاعدة البيانات.
- وشهدت قواعد بيانات المفتاح - قيمة انتشاراً واسعاً نظراً لأنها قابلة للتوسع بحيث يمكنها التعامل مع كميات هائلة من البيانات. فعدم استخدامها للتكامل المرجعي جعلها تسمح بكتابة وقراءة كميات كبيرة من البيانات بكفاءة. ويمكن تحسين سرعة المعالجة بشكل أكبر إذا تم تخزين أزواج المفتاح - قيمة في الذاكرة الرئيسية لقاعدة البيانات. وتسمى هذه الإعدادات بقواعد البيانات الموجودة في الذاكرة (In - memory Databases)، حيث يتم استخدام تقنيات تسمح بالتخزين المؤقت للقيم في الذاكرة الرئيسية مع التحقق المستمر من صحتها عن طريق مقارنتها مع البيانات المخزنة في الأقراص ذات التخزين طويل الأمد.
- وتتمتع قواعد بيانات المفتاح - قيمة بالمرونة فيما يتعلق بمخطط البيانات (Data Schema). ففي قواعد البيانات العلائقية، يجب بداية إنشاء مخطط شامل لبنية البيانات (المكونة عادة من جداول)، وربط مكوناته من خلال الواصفات (Attributes) قبل تخزين أي سجل في قاعدة البيانات. ولكن تخزين عدد كبير جداً من سجلات البيانات المتنوعة وغير المتجانسة يتطلب عملاً كثيراً ووقتاً طويلاً. أما قواعد بيانات المفتاح-قيمة فهي متسامحة مع ضرورة وجود مثل هذا المخطط، فلا ضرورة لإنشاء جدول بداخله أعمدة ذات أنماط بيانات محددة، وإنما يمكن مباشرة تخزين البيانات الموافقة لمفتاح ما. ولكن يجب الانتباه إلى أنه في كثير من الأحيان، يؤدي عدم وجود مخطط إلى فوضى في عملية إدارة البيانات.
- وتتمتع قواعد بيانات المفتاح - قيمة بميزة التجزئة الفعالة (Efficient Sharding) لكميات كبيرة من البيانات، فلا يوجد حد تقريباً لزيادة قابلية توسع قواعد بيانات المفتاح-قيمة مع استخدام تجزئة البيانات (Data Sharding). وعملية التجزئة سهلة نوعاً ما، فضمن عنقود الحواسيب (Cluster)، يتم توزيع

المفاتيح على الحواسيب المنضمة لنفس العنقود (تسمى Shards). ويتم بذلك توزيع قاعدة البيانات على عدد كبير من الأجهزة الفردية، وعادةً ما يتم توزيع المفاتيح وفقاً لمبادئ التقطيع المتسقة (Consistent Hashing).



الشكل 3: قاعدة بيانات المفتاح - قيمة موزعة على نطاق واسع مع تجزئة (Sharding) وتوزيع المفاتيح بالاعتماد على التقطيع (Hashing)²

ويوضح الشكل 3 بنية موزعة لقاعدة بيانات المفتاح-قيمة، حيث يتم توليد قيمة عددية كخرج لتطبيق تابع التقطيع (Hash Function) على المفتاح لتحديد على أي جزء (Shard) سيتم تخزين القيمة. ويتم توزيع هذه القيمة على عدد محدود من العناوين (تسمى Hash Slots)، ويمكن نسخ هذه القيمة على حواسيب أخرى، وتعديلها من أجل تحسين التسامح مع التجزئة. وهذا ما يُسمى بالنسخ المتماثل (Replication)، وتتم مزامنة البيانات الأساسية الموجودة في العنقود الأساسي (Master Cluster) مع بقية النسخ المكررة على بقية العناقيد التابعة (Slaves Clusters). ويضم العنقود الأساسي في الشكل 3 ثلاثة حواسيب (Shards) A و B و C. ويتم نقل البيانات مباشرة إلى الذاكرة الرئيسية لتقليل زمن الاستجابة. كما يتم إنشاء نسخة عن البيانات وإرسالها إلى عنقود تابع من أجل التخزين على وسط تخزين الدائم (أقراص تخزين)، ويتم إرسال نسخة أخرى إلى عنقود تابع آخر لتحسين الأداء، حيث تتم على هذا العنقود عمليات التحليل وتنفيذ الاستعلامات المعقدة.

² A. Meier and M. Kaufmann. (2019), **SQL & NoSQL Databases**, Springer Vieweg

4. قواعد بيانات عائلات الأعمدة:

على الرغم من أن قواعد بيانات المفتاح-قيمة قادرة على معالجة كميات كبيرة من البيانات بكفاءة، إلا أن هيكلها ما زال بدائياً. ففي كثير من الأحيان، تحتاج مصفوفة البيانات الحاوية على المفاتيح والقيم إلى أن تُنظَّم ضمن مخطط. وتعمل قواعد بيانات عائلات الأعمدة على تعزيز مفهوم المفتاح-قيمة من خلال توفير هيكل إضافي.

ومن الناحية العملية، أظهرت قواعد بيانات عائلات الأعمدة أنها أكثر كفاءة لتحسين عمليات القراءة من خلال تخزين سجلات البيانات في أعمدة الجداول، وبذلك تتميز عن قواعد البيانات العلائقية التي يتم فيها تخزين سجلات البيانات في أسطر الجداول. وتم الاعتماد على هذا الاختيار لأنه نادراً ما يوجد استعمال بحاجة إلى جميع أعمدة الصف الواحد، ولكن توجد مجموعات من الأعمدة تتم قراءتها معاً في أغلب الأحيان. فمن أجل تحسين الوصول إلى البيانات، من المفيد تنظيم البيانات ضمن مجموعات من الأعمدة، تُسمى عائلات الأعمدة، كوحدات تخزين.

وفي العام 2008، قدمت شركة Google النموذج Bigtable كقاعدة بيانات خاصة بها للتخزين الموزع للبيانات المنظمة، مما أثر بشكل كبير على تطوير قواعد بيانات عائلات الأعمدة. وفي هذا النموذج يمثل الجدول Bigtable خارطة متفرقة وموزعة ومتعددة الأبعاد ومفرزة (مصنفة). وتتمتع بالخصائص التالية:

- هيكل البيانات عبارة عن خارطة تقوم بربط عناصر من مجال إلى عناصر في مجال مشترك.
- يقوم تابع الخارطة (map function) بعملية فرز أيضاً، أي توجد علاقة ترتيب للمفاتيح التي ترتبط بالعناصر.
- العنونة متعددة الأبعاد، أي أن للتابع أكثر من بارامتر.
- يتم توزيع البيانات وفقاً للخارطة، أي يمكن تخزينها على أجهزة حاسوبية مختلفة في أماكن مختلفة.
- الخارطة متناثرة، حيث من المحتمل أن يكون هناك العديد من المفاتيح من دون إدخال عناصر بيانات مرتبطة بها.

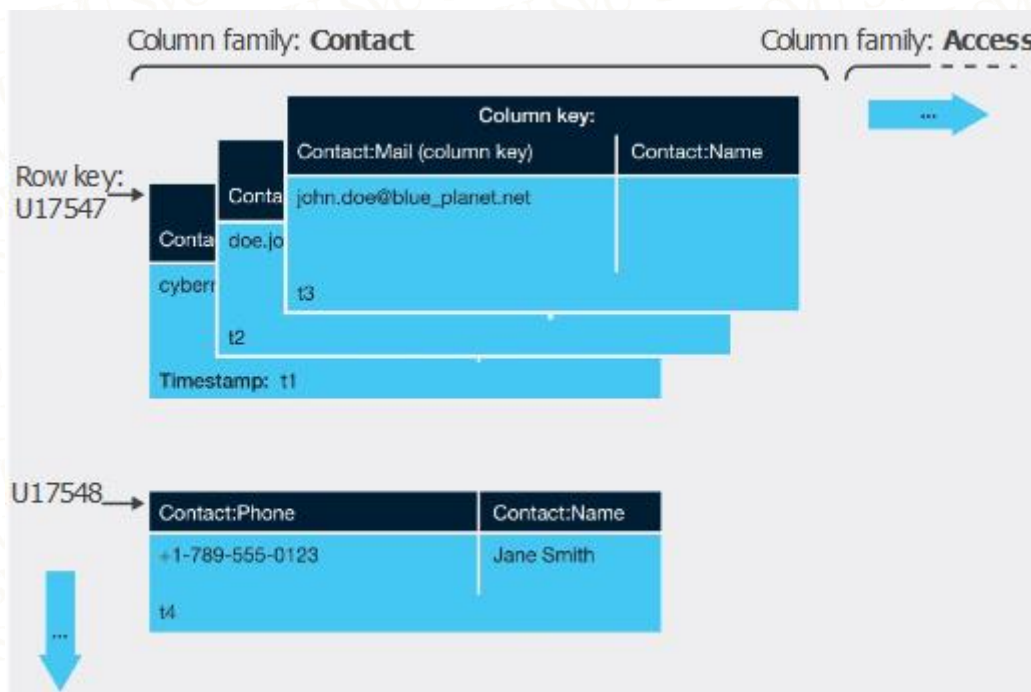
وال Bigtable ثلاثي الأبعاد، فهو يقوم بربط مدخل في قاعدة البيانات مع سلسلة نصية (string) مكونة من سطر (row) واحد وعمود (column) واحد وتوقيت (time):

string ← (row:string, column:string, time:int64)

ويتكون الجدول في قاعدة بيانات عائلات الأعمدة من بنية مجمعة متعددة المراحل. المرحلة الأولى متعلقة بالمفتاح، حيث تتم عنونة كل كائن بيانات بمفتاح كما في حالة قواعد بيانات المفتاح-قيمة. وضمن المفتاح توجد بنية أخرى، حيث يتم تقسيم السطر إلى عدة أعمدة، وتتم عنونة هذه الأعمدة من خلال مفاتيح أيضاً. أما بالنسبة للأسطر في الجدول، فيتم وضع طابع زمني (Time Stamp) لكل

إصدار (نسخة) من السطر الواحد. وتُسمى وحدة التخزين المكونة من مفتاح السطر ومفتاح العمود والطابع الزمني الموافق بالخلية (Cell)، ويتم تجميع الأعمدة في الجدول ضمن عائلات. وتمثل وحدة التحكم بالوصول، وذلك لضمان سماحيات القراءة والكتابة للأشخاص والتطبيقات. بالإضافة لذلك، لكل وحدة موافقة لعائلة أعمدة يتم تخصيص مساحة في الذاكرة الرئيسية ومساحة في القرص الصلب. وعائلة الأعمدة هي القاعدة الوحيدة الثابتة فيما يتعلق بمخطط الجدول. وبعبارة أخرى، قواعد البيانات العلائقية، يمكن استخدام عدة مفاتيح أعمدة ضمن عائلة أعمدة واحدة لتخزين البيانات. ولذلك، تمثل عائلة العمود مخططاً بدائياً مع كمية ضئيلة من البيانات الوصفية. وللبيانات ضمن عائلة الأعمدة نفس نمط المعطيات، حيث يُفترض أنها ستتم قراءتها معاً. وهذا هو السبب أيضاً في أن قاعدة البيانات تخزن دائماً بيانات عائلة أعمدة واحدة في صف واحد من الجدول على نفس الحاسوب. وتعمل هذه الآلية على تقليل الوقت اللازم للوصول إلى القراءة المجمعة داخل عائلة العمود. ولذلك، يقوم نظام إدارة قاعدة البيانات بفرز مجموعات الأعمدة في مجموعات محلية (Locality Groups) التي تحدد على أي حاسوب وبأي تنسيق سيتم تخزين البيانات.

ويتم تخزين بيانات مجموعة محلية واحدة فعلياً على نفس الحاسوب. بالإضافة إلى ذلك، من الممكن الاحتفاظ بمجموعة محلية معينة في الذاكرة الرئيسية؛ مما يجعل من الممكن قراءة البيانات بسرعة من دون الحاجة إلى الوصول إلى القرص الصلب.



الشكل 4: تخزين البيانات في Bigtable³

³ A. Meier and M. Kaufmann. (2019), **SQL & NoSQL Databases**, Springer Vieweg

ويوضح الشكل 4، كيفية تخزين البيانات في Bigtable، حيث تتم عنونة الخلية بواسطة مفتاح للسطر ومفتاح للعمود. وفي المثال الموضح في الشكل، يوجد مفتاح لكل مستخدم، ويتم توثيق المحتوى زمنياً باستخدام Timestamp. ونلاحظ وجود عائلة الأعمدة Contact التي تضم من الأعمدة: عمود البريد (Mail) وعمود الاسم (Name) وعمود رقم الهاتف (Phone). أما بيانات الوصول، مثل أسماء المستخدمين (User Names) وكلمات المرور (Passwords) فيمكن تخزينها ضمن عائلة الأعمدة Access. ونلاحظ أن الأعمدة في عائلة الأعمدة متفرقة، فالسطر U17547 على سبيل المثال يحوي العمود Contact:Mail ولا يحوي العمود Contact:Phone.

وتستخدم قواعد بيانات عائلات الأعمدة نموذجاً مشابهاً لـ Bigtable، وتتمتع بالخصائص الآتية:

- يتم تخزين البيانات في جداول متعددة الأبعاد.
- تتم عنونة كيانات البيانات بواسطة مفاتيح الأسطر.
- تتم عنونة خصائص كيانات البيانات بواسطة مفاتيح الأعمدة.
- يتم تجميع الأعمدة ضمن عائلات.
- مخطط الجدول يعبر فقط عن عائلات أعمدة، وضمن كل عائلة الأعمدة، يتم اختيار المفاتيح بطريقة عشوائية.
- في البنى الموزعة والمجزأة، من الأفضل تخزين بيانات كل عائلة أعمدة ضمن مكان واحد من أجل تحسين زمن الاستجابة.

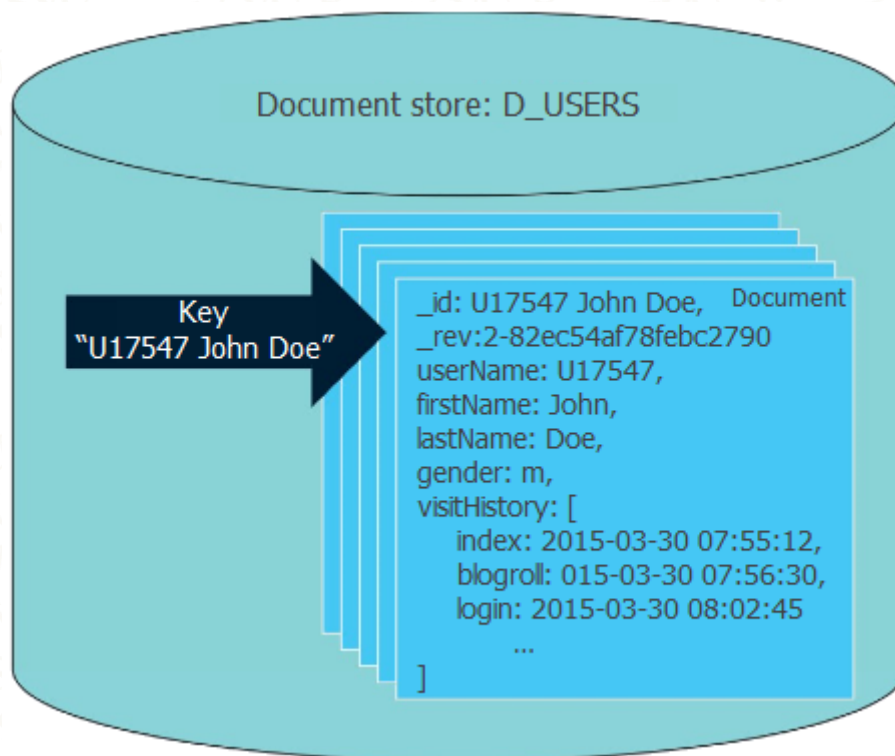
وتتمثل ميزات قواعد بيانات عائلات الأعمدة في قابليتها العالية للتوسع وتوفيرها الدائم نظراً لتوزيعها الهائل، تماماً كما هو الحال مع قواعد بيانات المفاتيح-قيمة. بالإضافة إلى ذلك، فإنها توفر بنية لها مخطط يوفر التحكم في الوصول وتحديد أماكن تخزين البيانات الموزعة على مستوى عائلات العمود، وفي نفس الوقت توفر مرونة كافية داخل عائلة الأعمدة الواحدة من خلال إتاحة استخدام مفاتيح أعمدة بطريقة عشوائية.

5. قواعد البيانات الموجهة بالمستندات:

المجموعة الثالثة من قواعد بيانات NoSQL هي قواعد البيانات الموجهة بالمستندات أو ما يُعرف بـ "مخازن المستندات"، وتجمع بين عدم وجود مخطط مع إمكانية هيكلة وتنظيم البيانات المخزنة. وعلى عكس مما يوحي به الاسم، لا تخزن قواعد البيانات الموجهة بالمستندات الملفات المتنوعة مثل ملفات الويب أو الفيديو أو البيانات الصوتية، ولكنها تحزن البيانات المنظمة ضمن سجلات تُسمى مستندات. وتم تطوير مخازن المستندات خصيصاً للاستخدام مع خدمات الويب (Web Services). ولذلك، يمكن دمجها بسهولة مع تقنيات الويب مثل JavaScript و HTTP. بالإضافة إلى ذلك، فهي قابلة للتوسع أفقياً بسهولة عن طريق الجمع بين العديد من الأجهزة الحاسوبية في نظام متكامل يوزع الحجم الكبيرة من البيانات عن طريق التجزئة (Sharding). وينصب التركيز في الغالب على معالجة كميات كبيرة من البيانات غير المتجانسة. فبالنسبة لمعظم بيانات الويب، على سبيل المثال بيانات وسائل التواصل الاجتماعي أو محركات البحث أو بوابات الأخبار، لا يلزم ضمان الاتساق المستمر للبيانات. وتُعتبر خدمات الويب الحساسة للأمن مثل الخدمات المصرفية عبر الإنترنت، والتي تعتمد بشكل كبير على قيود المخطط والاتساق المضمنون استثناءً لهذه القاعدة.

ولا تحتوي مخازن المستندات على مخطط (Schema)، أي لا حاجة لتعريف مخطط قبل إدخال البيانات. وبالتالي، يتم نقل مسؤولية وضع تصور عن بنية البيانات إلى المستخدم أو التطبيق الذي يتعامل مع البيانات. والعيب الناتج عن عدم وجود مخطط هو عدم وجود تكامل مرجعي وتسوية (Normalization). ومع ذلك، فإن عدم وجود مخطط يسمح بمرونة قصوى في تخزين مجموعة واسعة من البيانات، وهو ما تشير إليه خاصية التنوع (Variety) في البيانات الضخمة، ويسهل أيضاً تجزئة البيانات.

وفي المستوى الأول، تعد مخازن المستندات نوعاً من قواعد بيانات القيمة-مفتاح. ولكل مفتاح (معرف المستند)، يمكن تخزين سجل كقيمة. وتسمى هذه السجلات "مستندات". وفي المستوى الثاني، يكون للسجلات هيكلها الداخلي الخاص. ومصطلح "مستند" ليس دقيقاً تماماً، نظراً لأنه لا يشمل ملفات الوسائط المتعددة والبيانات الأخرى غير المنظمة. فالمستند المقصود ضمن سياق مخازن المستندات هو ملف فيه بيانات منظمة وفقاً لتنسيق JSON، وبنيتها عبارة عن قائمة من الأزواج خاصية-قيمة. ويمكن أن تحتوي جميع الأزواج خاصية-قيمة في بنية البيانات هذه بشكل متكرر على قوائم لنفس الأزواج. والمستندات ليست متصلة ببعضها البعض، ولكنها تحتوي على مجموعة مغلقة من البيانات.



الشكل 5: مثال عن مخازن المستندات⁴

ويوضح الشكل 5 مثلاً عن نموذج مخزن المستندات، حيث D_USERS هو مخزن مستندات لتخزين بيانات مستخدم موقع ويب. ويوجد لكل مفتاح خاص بمستخدم معرف (قيمة للوصف _id)، وفي كل مستند (كائن) يتم تخزين جميع معلومات المستخدم، مثل الاسم الأول واسم العائلة والجنس. وتحتوي الوصفة visitHistory على واصفة متداخلة فتصبح مشابهة لمصفوفة ترابطية والتي تحتوي مرة أخرى على أزواج مفتاح - قيمة، وتسرد هذه البنية المتداخلة تاريخ الزيارة الأخيرة إلى موقع الويب على أنها القيمة المرتبطة.

وبالإضافة للوصف _id، يحتوي المستند على حقل _rev (revision) الذي مهمته فهرسة إصدار المستند. وأحد الاحتمالات لحل الاستعلامات المتزامنة هو التحكم المتزامن متعدد الإصدارات، حيث تقوم قاعدة البيانات بالتأكد من أن كل استعلام يتلقى مراجعة المستند الذي يحتوي على أكبر عدد من التغييرات. وبما أن هذه العملية لا يمكن أن تضمن الاتساق الآني للمعاملات، يُطلق عليها الاتساق النهائي، ولا يتم الوصول إلى اتساق البيانات إلا بعد مرور بعض الوقت.

A. Meier and M. Kaufmann. (2019), **SQL & NoSQL Databases**, Springer Vieweg ⁴

ويمكن تلخيص خصائص قواعد البيانات الموجهة بالمستندات بما يأتي:

- هي مخازن من نوع مفتاح - قيمة.
 - يتم تخزين البيانات ككائنات (Objects)، وتمثل الكائنات قيماً متوافقة للمفاتيح وتُسمى مستندات (Documents)؛ كما يتم استخدام المفاتيح كمعرفات.
 - تضم المستندات بنى معطيات على شكل أزواج من الواصفات المتداخلة (Nested Attributes) من دون تكامل مرجعي.
 - بنى المعطيات المستخدمة ليس لها مخطط ثابت، حيث يتم استخدام الواصفات ضمن كل مستند وإسناد قيم لها مباشرة من دون الحاجة إلى تعريف مخطط لتوضيح بنية المستند.
- ويمكن للاستعلامات الموجهة لمخزن المستندات أن تتم معالجتها على التوازي وتسريعها من خلال إجرائية MapReduce المكونة من مرحلتين؛ في الأولى يقوم التابع Map بإنشاء مجموعات من المستندات، وفي الثانية يقوم التابع Reduce بعمليات الـ Aggregation مباشرة على المستندات المجمعة، حيث يمكن تنفيذ عمليات count و sum وغيرها.
- وخلال المرحلة الأولى، يتم استخدام التابع map الذي ينفذ عملية مسبقة على كل مستند، حيث يقوم بإنشاء خارطة map، وهي عبارة عن مصفوفة ترابطية فيها زوج واحد أو عدة أزواج من المفتاح - قيمة لكل مستند. ويمكن لهذه المرحلة أن تُطبق على كل مستند بشكل مستقل عن بقية البيانات؛ وهذا ما يسمح بالمعالجة المتوازية من دون حدوث تداخل بين العمليات في حال كانت قاعدة البيانات موزعة على عدة حواسيب. أما في المرحلة الاختيارية Reduce، فيتم تنفيذ تابع لتقليل البيانات من خلال إرجاع سطر واحد لكل مفتاح من الفهرس الناتج عن التابع map، وتجميع القيم المتوافقة. ويوضح المثال الآتي كيفية عمل Map/Reduce لحساب عدد المستخدمين الذين يتم تجميعهم وفقاً للجنس (gender):

```
// map
function(doc){
  if(doc.userName) {
    emit(doc.gender, 1)
  }
}

// reduce
function(keys, values) {
  return sum(values)
}

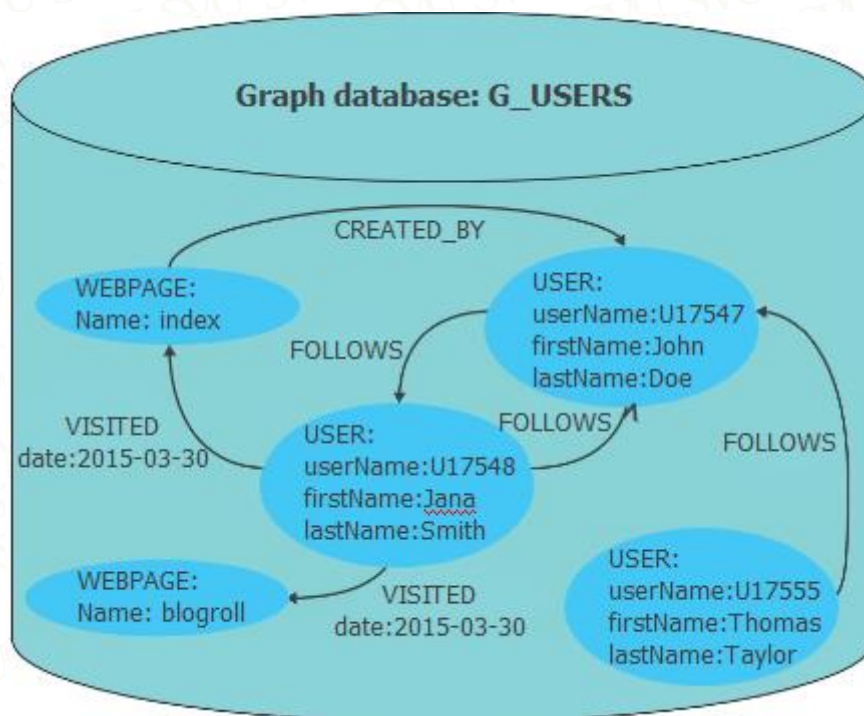
// > key value
// > "f" 456 // > "m" 567
```

وبسبب غياب المخطط، يقوم التابع map بالتنفيذ من أجل كل مستند للتحقق من أن الوصفة username موجودة أم لا. وفي حال كانت موجودة، يقوم التابع emit بإرجاع زوج المفتاح-قيمة، ويكون المفتاح هو جنس المستخدم، والقيمة هي 1. ويستلم التابع reduce مفاتيح آخرين، m و f، ضمن مصفوفة المفاتيح keys. ومن أجل كل مستند خاص بمستخدم موافق للجنس المطلوب، يتم وضع العدد 1 كقيمة في مصفوفة القيم values. ويقوم التابع reduce بإرجاع مجموع الواحدات مجمعة وفقاً للمفاتيح.

ونتائج عمليات Map/Reduce تسمى مناظير (Views)، ويجب حسابها مسبقاً وفهرستها لتصبح دائمة من أجل تحسين الأداء. ويتم تخزين الثنائيات مفتاح-قيمة في بنية معطيات شجرية من النمط B-Tree. وهذا ما يسمح بالوصول السريع إلى قيم كل مفتاح على انفراد. ويستخدم التابع map بنية المعطيات الشجرية لتخزين التجميعات في أشجار متوازنة مع قيم تفصيلية قليلة مخزنة في الأوراق. وعند تعديل التجميعات، يجب فقط تعديل الورقة المعنية وبعض العقد التي تضم المجاميع الجزئية وهكذا حتى الوصول إلى الجذر.

6. قواعد بيانات الرسم البياني:

يختلف النوع الرابع والأخير من قواعد بيانات NoSQL الأساسية بشكل كبير عن الأنواع المذكورة سابقاً، أي قواعد بيانات المفتاح-قيمة وقواعد بيانات عائلات الأعمدة وقواعد البيانات الموجهة بالمستندات. وتتخلى نماذج البيانات الثلاثة هذه عن مخططات قواعد البيانات والتكامل المرجعي من أجل تسهيل التجزئة (Sharding). ومع ذلك، تحتوي قواعد بيانات الرسم البياني على مخطط هيكلي. وفي قاعدة بيانات الرسم البياني، يتم تخزين البيانات كعقد ووصلات، ولكل منها نمط معطيات، وتحتوي على بيانات على شكل أزواج واصفة - قيمة (attribute-value). وعلى عكس قواعد البيانات العلائقية التي يكون مخططها ضمنياً، يمكن في قواعد بيانات الشكل البياني إدراج كائنات البيانات مباشرة في قاعدة البيانات على الرغم من أن لها نمط معطيات غير معرف مسبقاً. ويتتبع نظام إدارة قواعد البيانات (DBMS) ضمناً التغييرات التي تطرأ على المخطط بناءً على المعلومات المتاحة وبالتالي يقوم بإنشاء نمط المعطيات المعني. ويوضح الشكل 6 مثلاً عن قاعدة بيانات الرسم البياني G_USERS التي تحتوي بيانات حول منصة ويب لها مستخدمين وصفحات ويب وعلاقات بينهم. ويوجد نوعان من العقد USER وWEBPAGE وثلاثة أنواع من الوصلات FOLLOWS وVISITED وCREATED_BY. وعقد الـ USER لها الواصفات username وlastName. أما العقدة WEBPAGE، فلها الواصفة Name، وتمتلك الوصلة VISITED واصفة واحدة date. وبالتالي، فهو رسم بياني خاص بالواصفات.



الشكل 6: مثال عن قاعدة بيانات الرسم البياني التي تضم معلومات عن مستخدم موقع ويب⁵

⁵ A. Meier and M. Kaufmann. (2019), **SQL & NoSQL Databases**, Springer Vieweg

وفي الشكل 6، يضم الرسم البياني أنماط معطيات متشابهة مثل قاعدة بيانات المستندات D_USERS الموضحة في الشكل 5، وعلى سبيل المثال، يوضح أسماء المستخدمين وأسماء كنياتهم والصفحات التي تمت زيارتها وتواريخ الزيارات. ولكن يوجد فرق هام يتمثل في أن العلاقة بين كيانات البيانات ممثلة بوضوح من خلال الوصلات، وبالتالي يوجد تكامل مرجعي يضمه نظام إدارة قواعد البيانات. وتتمتع قواعد بيانات الرسم البياني بالخصائص الآتية:

- يتم توضيح البيانات و/ أو المخطط من خلال رسم بياني (Graph) أو بنية تشبه الرسم البياني.
- يتم تمثيل معالجة البيانات من خلال تغيرات على الرسم البياني، أو من خلال عمليات تؤثر في خصائص الرسم البياني (مثل، المسارات، التجاور، البيانات الجزئية، الاتصالات بين العقد، وغيرها).
- تدعم قاعدة البيانات التحقق من قيود التكامل المرجعي من أجل ضمان اتساق البيانات. ويتعلق مفهوم اتساق البيانات مباشرة ببنية الرسم البياني (مثل، أنماط العقد والوصلات، مجال الواصفات، التكامل المرجعي للوصلات).

وتُستخدم قواعد بيانات الرسم البياني عندما يتم تنظيم البيانات في شبكات. وفي هذه الحالة، لا يتم التركيز على سجل بيانات فقط، وإنما على ارتباطاته مع بقية السجلات الأخرى، أي من المهم التركيز على العلاقات التي تربط كيانات البيانات، كما هو الحال في شبكات التواصل الاجتماعي، وعند تحليل بيانات الوصلات بين مواقع الويب، وعند تحليل بيانات شبكات البنى التحتية للمدن (مثل، شبكات المياه والكهرباء).

وميزة هذا النوع من قواعد البيانات تكمن في أنه لكشف التقارب بين العقد لا داعٍ للحصول على فهرس (index-free)، فنظام قواعد البيانات يمكنه مباشرة اكتشاف العقدة المجاورة لكل عقدة مباشرة من دون الحاجة إلى المرور في عدد كبير من الوصلات. ولذلك، يكون الجهد المبذول للاستجابة لاستعلام ما تقريباً ثابتاً ومستقلاً عن حجم البيانات. وإذا ما قارنا هذا الأمر بقواعد البيانات العلائقية، نجد أنها تعتمد التكامل المرجعي الذي يفرض المرور على عدة جداول وربطها مع بعض من خلال العملية Join من أجل الاستجابة على استعلام ما، وكلما كبر عدد الجداول الداخلة في الاستعلام وعدد سجلات البيانات المطلوب معالجتها سيزداد الجهد المبذول للاستجابة للاستعلام.

وقواعد بيانات الرسم البياني تشبه قواعد البيانات العلائقية من ناحية الحاجة إلى فهرس لتحقيق وصول سريع ومباشر للعقد المفردة والوصلات من خلال واصفاتها. وتستخدم أشجاراً متوازنة (B-trees) من أجل الفهرسة. والشجرة هي بيان خاص لا يحتوي على أي حلقة، وكل شجرة يتم تمثيلها ببيان. وهذا مفيد لقواعد بيانات الرسم البياني لأن الفهرس في هذه الحالة سيكون بيان-جزئي من البيان الأساسي، وبذلك يحتوي البيان على الفهارس الخاصة به.

ولكن عملية تقسيم البيان معقدة نوعاً ما، وأحد الأسباب التي جعلت قواعد بيانات NoSQL الأخرى لا تركز على العلاقات بين السجلات هو الحاجة إلى المحافظة على إمكانية تخزين البيانات على عدة حواسيب مع أقسامها من دون الاهتمام بالارتباطات بينها. وهذا معاكس تماماً لحالة قواعد بيانات الرسم البياني، فهي تركز كثيراً على العلاقات البينية بين السجلات وإنشاء هذه العلاقات تحتاج إلى خبراء في المجال المعرفي للبيانات المخزنة. ولا توجد في الحقيقة طريقة مثلى لتجزئة بيان إلى بيانات جزئية. وجميع الخوارزميات المتاحة هي NP-complete. وكطرق تجريبية (Heuristic)، يمكن لخوارزميات العنقدة تقسيم البيان إلى بيانات جزئية من خلال تحديد الارتباطات القوية بين كل مجموعة عقد ووضعها ضمن عنقود يمثل بياناً جزئياً. ونتيجة لهذه المشكلة، معظم قواعد بيانات الرسم البياني لا تدعم تجزئة البيانات (Sharding).

7. تمارين:

1. ما أهم خصائص قواعد بيانات NoSQL؟
2. ما هي العيوب التي تعاني منها قواعد بيانات NoSQL؟
3. ما الفرق بين قواعد بيانات NoSQL وقواعد البيانات العلائقية؟
4. ما الفرق بين نموذج اتساق ACID ونموذج اتساق BASE؟
5. على ماذا تنص نظرية CAP؟
6. عدد أشهر أنواع قواعد بيانات NoSQL وقارن بينها
7. متى يجب استخدام قواعد بيانات NoSQL؟
8. ما المقصود بقواعد بيانات المفتاح-قيمة؟
9. ما المقصود بقواعد البيانات الموجهة بالمستندات؟
10. ما المقصود بقواعد البيانات عائلات الأعمدة؟
11. ما المقصود بتجزئة البيانات (Data Sharding) في قواعد بيانات NoSQL؟
12. اشرح مفهوم التنسيق النهائي المتبع في قواعد بيانات NoSQL
13. متى يكون استخدام قواعد بيانات الرسم البياني خياراً جيداً؟
14. ما هي ميزات قواعد بيانات الرسم البياني؟
15. ما هي العيوب التي تعاني منها قواعد بيانات الرسم البياني؟